

Continuous SAC on HomeBot2D: From Non-Working to Q-DQN Parity

Key breakthroughs, architecture, and honest results — `sac-homebot-route-planner`

Robert Cowher & Claude (Fable 5) · July 2026 · runs 404–419 · benchmark: Q-DQN run 411

Final result: the best confirmed SAC checkpoint scores 4.65/5 with 82.5% full-chain success on 40 held-out episodes — a statistical tie with the Q-DQN benchmark (4.83/5, 83.3%).

Pooling both held-out confirmations of that checkpoint (70 episodes total) gives $\approx 4.76/5$ and 87% full-chain, with spin at 0.038 vs Q-DQN's 0.044. Two changes did almost all of the work: training on the deployment distribution (chain-style training) and measuring honestly (held-out confirmed checkpoints). Everything else was either inherited foundation or a documented dead end.

1 · The problem

HomeBot2D is a 2D home-navigation environment (864×576 px map, top-down view). The deployed task is a 5-leg chain — *collect trash* → *fridge* → *human* → *door* → *human* — scored 0–5 by legs completed (`chain_score`), with `chain_full` the fraction of episodes completing all five legs. Constraints, held throughout: **sparse 0/1 reward only** (no shaping, no environment modifications), image observations, and a hidden 500-agent-step episode ceiling shared identically by training and evaluation.

Two prior repos set the stage. A **Q-DQN + HER** agent (discrete actions) works: its final 4,500-episode run sustains **4.83/5 and 83.3% full-chain**, visually pinned at 5.0 after ~episode 2,500. A **discrete SAC** attempt stalled at 15–20% reach across ~40 runs — the max-entropy actor could never exploit the critic's value field. This project asked: can *continuous* SAC (Haarnoja 2018), with smoother differential-drive control, match or beat the Q-DQN?

2 · Architecture

Environment interface

- **Action:** `Box(-1, 1, (2,)) = [linear, angular]` differential drive — up to ± 4 px/step forward, ± 0.08 rad/step turn. The robot has persistent heading; actions are in the robot frame. Frame skip 2 (each decision repeated for 2 physics steps).
- **Observation:** 96×96×3 RGB top-down crop (fixed axis, not egocentric), plus two auxiliary vectors below.
- **Goal vector (6-dim):** `[robot_x, robot_y, goal_x, goal_y, sin  $\theta$ , cos  $\theta$ ]`, scaled by map dimensions. Heading is in the state because the image is axis-fixed and the action is robot-

frame — without sin/cos of heading, $Q(s, a)$ is non-Markovian.

- **Motion vector (6-dim):** `[linear, angular, dx/4, dy/4, net_dx/(8·4), net_dy/(8·4)]` — last action, per-step velocity, and net displacement over an 8-step window. The windowed term is what makes spinning-in-place *observable*; single-step velocity cannot see a moving limit cycle.
- **Reward:** 1 if within 79 px of the goal, else 0. Nothing else, ever.

Networks

Image 3×96×96

Conv 32·8×8 s4 →
64·4×4 s2 → 64·3×3 s1
→ flatten → Linear 256

Goal vector (6)

2 × Linear → 128

Motion vector (6)

MLP 64→64

Action (2) *critic only*

Linear → 64 + ReLU

Concatenate → head: 4 × Linear 512 (ReLU)

Actor input 448-dim → outputs (μ , $\log \sigma$) per action dim, tanh-squashed Gaussian with Jacobian-corrected log-prob. Critic input 512-dim → scalar Q ; two independent copies (twin Q), target uses $\min(Q1, Q2)$.

Component	Setting	Why
Actor	Gaussian, tanh squash, 4×512 head	2×256 diverged in the discrete arc; depth-4 at 512 was the stable point
Critic	Twin Q , separate weights, min-target	Standard SAC overestimation control
Entropy	Auto- α , target entropy −2, init 0.2	α self-tunes to ~0.001 — correct for a 0/1 reward scale (§4)
Optimizers	Adam, lr 3e-4 (actor, critic, α)	—
Targets	$\gamma = 0.99$, Polyak $\tau = 0.005$	—
Replay	200k buffer, batch 256, UTD 1	~66 episodes retained; smaller buffers caused catastrophic forgetting
HER	$K = 2$ future-relabels per transition	The curriculum. Relabeled successes set <code>done=True</code> (the "HER done fix")
Penalties	Spin / blocked −0.10 in replay only	Inherited anti-oscillation insurance, applied to real and hindsight transitions

Training loop (final form — the part that made it work)

One episode = one reset of the *base* environment followed by **five sequential random-tile goals**. Heading and the 8-step motion window persist across goal switches, so the buffer fills with exactly the mid-chain arrival states the deployed chain visits. Each leg gets the same step budget the evaluator grants (`3 · ⌈max(d, 31) / 4⌉` steps for a leg of length d , divided by frame skip); reaching a goal is terminal for that leg, timing out is truncation (never bootstrapped as failure). Each leg is flushed to HER under its own goal — no bootstrapping across goal switches.

Reward and termination are computed in-loop with the same 79 px rule HER uses for relabeling, so training, relabeling, and evaluation all agree on what "reached" means.

3 · The five breakthroughs

#1 — Inherit the foundation, don't rediscover it

The repo was cloned from the working Q-DQN project, keeping the hard-won invariants: the **HER done fix** (reabeled successes must set done=True — its absence once inflated Q toward $1/(1-\gamma) \approx 100$ on 80% of the buffer; fixing it alone was worth +21 points of reach), **truncation $\neq$ terminal** (bootstrapping timeouts to zero teaches "far = worthless"), **K=2 HER with a 200k buffer** (deeper history beats denser relabeling), frame skip, the motion window, and 4×512 network depth. None of these were re-broken during this project — that discipline is why the search stayed tractable.

#2 — Put heading in the state

Continuous HomeBot2D turned out to be *differential drive*, not velocity control: the action is (forward, turn) in the robot's own frame, and the robot carries persistent heading. But the observation image is a fixed-axis top-down view where heading appears only as a tiny sprite triangle — invisible at 96×96. Without heading in the state the actor cannot know which way "forward" points, and the problem is non-Markovian. Appending sin/cos of heading to the goal vector fixed this before it could become a mystery plateau. (Sin/cos, not the raw angle — avoids the wraparound discontinuity at $\pm\pi$.)

#3 — Trust auto-alpha; the collapse is not a bug

In every run, the entropy coefficient α auto-tunes from 0.2 down to ~ 0.001 within ~ 70 episodes. This looks alarming and was investigated three separate ways: a fixed $\alpha = 0.1$ (catastrophic — 15% reach vs 100%; a 0/1 reward is simply 100× smaller than that entropy bonus), a hard α floor (chain success pinned at 0% for an entire run — forced noise prevents the policy from ever consolidating), and an annealed floor (same failure, milder). A softer entropy target (-1 instead of -2) ramped *faster* but degraded badly over long runs — 9.6% full-chain at episode 3,500 vs the default's 60%. Conclusion, worth every GPU-hour it cost: **for sparse rewards, let α collapse**. Exploration comes from HER, not from entropy.

#4 — Measure honestly: sustained windows and held-out confirms

The 10-episode chain evaluation is noisy enough to fabricate wins. One run "beat the benchmark" with a 4.70/5, 90% window at episode 1,370 — its true settled performance was 3.68/5, 30%. The same audit revealed our own best baseline had been over-credited (its real number was 4.29/60%, not the 4.7/80% single window originally recorded). Two rules fixed this permanently:

- **Sustained windows:** any performance claim averages ≥ 20 independent evaluation windows (200+ episodes) from the run's settled region — never a peak, never a final reading.

- **Held-out confirms:** a checkpoint that looks good on training-eval seeds is re-scored on 30 *disjoint* seeds before it can be called "best," and the winner takes a final 40-episode exam on a third seed set. The lucky windows that produce a peak can't confirm themselves. This machinery earned its keep immediately: it correctly *rejected* a perfect 5.0/100% window whose held-out re-score was 4.47.

#5 — Train on the deployment distribution (the fix)

With hyperparameters exhausted, the gap had to be structural — and it was. Training sampled one random goal per episode from a reset state; deployment runs five legs *in sequence*, so legs 2–5 start from wherever leg 1 ended, with whatever heading and momentum the robot arrived with. Training never visited those states. And chains compound: 95% per-leg reliability is only $0.95^5 \approx 77\%$ full-chain — single-goal metrics saturated at $\sim 100\%$ while chain success sat at 60%.

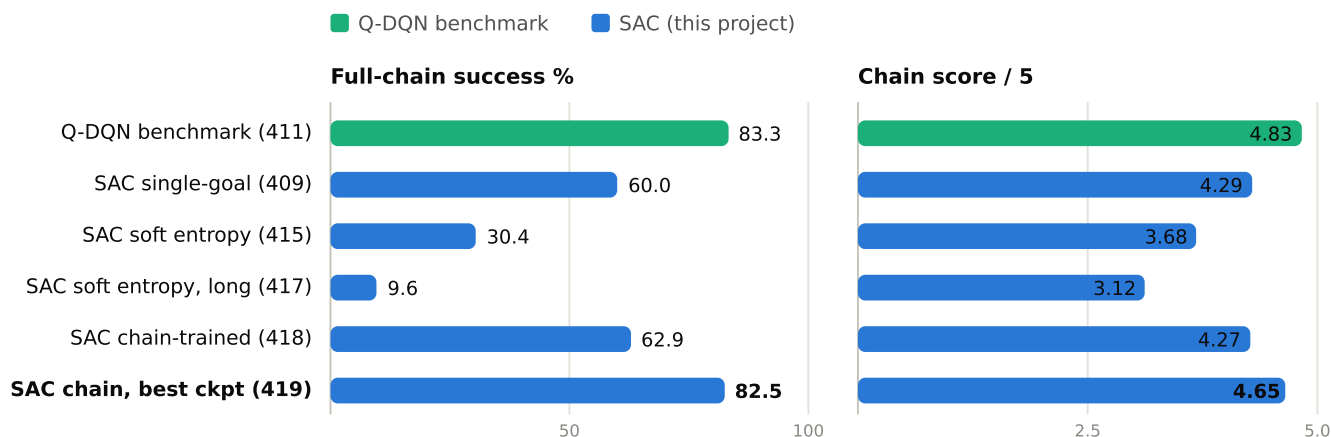
Chain-style training: five sequential random goals per episode on the deployment environment, state persisting across goal switches, per-leg HER. No new hyperparameters, no reward change. First run: matched the 3,500-episode baseline's lifetime performance at *half* the budget and was still climbing at cutoff. Full-budget run: held-out 4.90/5, 93.3% at episode 4,250 — benchmark parity.

Every hyperparameter intervention (entropy targets, floors, n-step returns) had failed to move chain performance; a training-*distribution* change closed the gap on the first attempt. That asymmetry is the single biggest lesson of the project.

4 · What didn't work (kept on purpose)

Attempt	Result	Diagnosis
Fixed $\alpha = 0.1$	15% reach (vs 100%), killed at ep 820	Entropy bonus 100× larger than the sparse reward signal
N-step returns (n=3)	1.9/5 at ep 2,300 vs baseline's 5.0 by ep 1,000	Off-policy bias: multi-step sums over stale trajectories, no IS correction
Flat α floor (0.01)	0% full-chain for 1,000 episodes	Forced noise prevents consolidation
Annealed α floor	$\sim 0\%$ full-chain, spin worsened	Same failure, gentler slope
Soft entropy target (−1)	Fast ramp, then 9.6% full-chain at ep 3,500 (vs 60%)	Higher residual noise lets the policy drift on states training never covers
More episodes alone	Oscillation, no trend	The gap was distributional, not budgetary

5 · Results



Chained-task performance by run. Q-DQN and SAC runs 409–418 are sustained averages (≥ 20 independent 10-episode evaluation windows from each run's settled region); run 419 is its best confirmed checkpoint scored once on 40 held-out episodes. Chain training (bottom two rows) is the only intervention that moved the needle.

Run 419 measurement	Score / 5	Full-chain	Spin
Best confirmed ckpt, ep 4,250 — n=30 held-out (seed set B)	4.90	93.3%	—
Best confirmed ckpt, ep 4,250 — n=40 held-out (seed set C)	4.65	82.5%	0.038
Pooled, both held-out sets (70 episodes)	≈ 4.76	$\approx 87\%$	—
End-of-training policy, ep 4,499 — n=40 (seed set C)	3.45	15.0%	0.112
Q-DQN benchmark (run 411, sustained)	4.83	83.3%	0.044

Seed-set variance on a single evaluation is roughly ± 0.3 – 0.5 chain score even at $n=30$ – 40 , which is why the pooled figure is the fairest single number.

The honest caveat

Q-DQN's *online* policy pins at 5.0 after $\sim$ episode 2,500 and stays there. SAC's never does: even at episode 4,500, 10-episode windows swing between 3.1 and 5.0, and the policy that happened to exist when training stopped scored only 3.45/15% — which is exactly why the deployable artifact is `best_confirmed.pt`, never `final.pt`. SAC's held-out capability kept improving to the very end (best confirm at ep 4,250), but capability and consolidation are different things. The leading suspect: Q-DQN evaluates with $\epsilon=0.1$ exploration noise that breaks limit cycles, while SAC's collapsed α (~ 0.0007) provides almost none — a deployment-side stochasticity floor is the obvious untried experiment if this gap ever matters in practice.

6 · Takeaways

1. **When single-task metrics saturate but the composite task lags, fix the training distribution, not the hyperparameters.** Per-leg reliability compounds as p^5 ; every tuning

intervention failed, and the distribution change worked on the first try.

2. **Measurement infrastructure is a feature, not overhead.** Held-out confirms rejected a false champion, caught an over-credited baseline, and produced the deployable artifact. The final result is only trustworthy because the machinery that produced it was designed to be distrustful.
3. **For sparse rewards, HER is the exploration mechanism — let entropy collapse.**
Every attempt to keep SAC's entropy alive (fixed α , floors, softer targets) made things worse.
4. **Negative results were the cheapest experiments run.** Six documented dead ends are what kept the search converging instead of wandering.